Activation Service

Manual do Desenvolvedor: visão técnica do projeto para quem vai avaliá-lo ou mantê-lo

Este documento resume o que foi construído, onde cada item do desafio está atendido e por que as principais decisões foram tomadas. O detalhamento completo está no repositório: `README.md`, `docs/spec` (requisitos, design e plano) e `docs/adr` (quatro registros de decisão).

Java 21
Spring Boot 3.5

185
testes automatizados

100%
linhas e ramos (gate no build)

4
ADRs

SDD
especificação aprovada antes do código

Sumário

1. Visão geral
2. Como avaliar em 5 minutos
3. Mapa do desafio para a solução
4. Arquitetura
5. Ciclo de vida de uma solicitação
6. Decisões técnicas e trade-offs
7. Estratégia de testes
8. Operação e produção
9. Limitações conhecidas e próximos passos
10. Guia de leitura do código
11. Processo de desenvolvimento e uso de IA

1. Visão geral

API REST que recebe pedidos de ativação de produto, consulta o status e realiza a ativação num provedor externo. A solução se apoia em três decisões centrais:

- **Duplicidade garantida pelo banco:** uma constraint única em `external_id` decide qualquer corrida; o retry do sistema de origem recebe `200` com o registro existente, e conflitos recebem `409`.
- **Integração assíncrona e resiliente:** o `POST` grava `PENDING` e responde na hora; um worker usa a própria tabela como fila (`FOR UPDATE SKIP LOCKED`), chama o provedor fora de transação e trata cada resposta, com backoff exponencial, jitter, `Retry-After`, chave de idempotência, reconciliação de trabalho abandonado e circuit breaker.
- **Segurança implementada:** OAuth 2.0 client credentials com JWT validado (assinatura, emissor, audiência, expiração), escopos de leitura e escrita, isolamento entre sistemas consumidores e limite de chamadas por sistema.

Camada	Tecnologia
Linguagem e framework	Java 21 (records, sealed interfaces, pattern matching) · Spring Boot 3.5 (Spring MVC, Spring Data JPA/Hibernate, Spring Security)
Persistência	PostgreSQL 16 · Flyway
Integração	`RestClient` sobre o `HttpClient` do JDK, com timeouts explícitos
Testes	JUnit 5 · Mockito · AssertJ · MockMvc · Testcontainers · WireMock · Awaitility · ArchUnit · JaCoCo
Resiliência	Resilience4j (circuit breaker e limite de chamadas)
Operação	Actuator · Micrometer/Prometheus · OpenTelemetry/Jaeger · logs JSON (ECS) · Docker multi-stage · Docker Compose · GitHub Actions
Documentação da API	OpenAPI (springdoc) · Swagger UI com login OAuth2 · coleção Postman

2. Como avaliar em 5 minutos

Pré-requisitos: JDK 21 e Docker em execução.

1. **Build e testes** (unitários, integração com PostgreSQL real, arquitetura e cobertura):

   ```
   ./mvnw verify
   ```

2. **Ambiente completo** (aplicação, PostgreSQL, provedor simulado, servidor OAuth2 e Jaeger, em `http://localhost:18686`):

   ```
   docker compose up -d --build
   ```

3. **Explorar a API** em `http://localhost:18080/swagger-ui.html`: **Authorize** (credenciais de demonstração pré-preenchidas) → **select all** → **Authorize**, e então **Try it out** em qualquer operação.
4. **Ou rodar a coleção Postman** `docs/postman/activation-service.postman_collection.json`: 16 requisições com o resultado esperado verificado automaticamente.

Para observar os cenários de falha, o provedor simulado responde conforme o `productCode`: `INVALID-PRODUCT` (recusa 400), `UNAVAILABLE-PRODUCT` (503 com `Retry-After`), `SLOW-PRODUCT` (acima do timeout). As portas ficam na faixa 18xxx por padrão e podem ser trocadas por variável de ambiente (ver README).

3. Mapa do desafio para a solução

Cada item do enunciado, como foi atendido e onde verificar. `Impl.` = implementado e testado · `Desc.` = descrito no README.

Item do desafio		Como foi atendido	Onde verificar
1. Tecnologias	Impl.	Java 21, Spring Boot 3.5, Maven Wrapper, PostgreSQL, Flyway, Compose, CI; records e sealed + switch	`pom.xml`, `db/migration`, `.github/workflows`
2. POST e GET	Impl.	Validação, `201` + `Location`, datas UTC ISO-8601, DTOs separados da entidade, `404`	`ActivationRequestController` · `ActivationRequestControllerTest`, `ActivationRequestApiIT`
2. Listagem paginada	Impl.	Filtros por status e cliente, paginação, índices justificados	`V1__...sql`, `design.md` §4 · `ActivationRequestListingIT`
3. Duplicidade	Impl.	Constraint única; 200 para retry, 409 para conflito, 409 genérico entre clientes; explicação de por que verificar antes não basta	ADR-0002 · `DuplicateRequestConcurrencyIT` (20 threads)
4. Estados e transições	Impl.	Transições só por métodos do domínio; falha temporária volta para `PENDING`; `@Version`	`ActivationRequest` · `ActivationRequestTest`
5. Integração	Impl.	Tabela de retorno (2xx, 400, 401/403, 500, timeout, 429/503); assíncrono; sem HTTP dentro de transação; envio garantido mesmo com queda; timeouts configuráveis; `Idempotency-Key`; backoff com jitter e limite; reconciliação	ADR-0001, ADR-0003 · `ActivationProcessingIT`, `ProviderClientTest`
5. Circuit breaker	Impl.	Resilience4j; abre com falhas do provedor (5xx, timeout, credencial recusada), não com pedidos recusados; aberto, nada sai da fila e nenhuma tentativa é gasta	ADR-0003 · `ActivationWorkerTest`, `ActivationProcessingIT`
6. Tratamento de erros	Impl.	RFC 9457 em todos os erros, inclusive 401/403/405/413/415/429; `traceId` e `correlationId`; nenhum detalhe interno exposto	`ApiExceptionHandler` · `ApiExceptionHandlerTest`
7. Segurança	Impl.	Resource Server, escopos distintos, 401/403 testados sem servidor real, isolamento contra BOLA, limite de corpo e limite de chamadas por cliente (`429`)	ADR-0004 · `JwtValidationIT`, `ActivationRequestControllerTest`, `RateLimitTest`
8. Testes	Impl.	Todos os cenários mínimos e todos os diferenciais da tabela: concorrência, timeout, domínio, segurança, ArchUnit	Seção 7 deste manual
9. Qualidade	Impl.	Pacotes com regras verificadas; commits pequenos, cada um com a intenção e a tarefa do plano citadas (tarefa → requisito em `tasks.md`)	`ArchitectureTest` · `git log`

Observabilidade	Impl.	Liveness e readiness, métricas Prometheus, logs JSON, tracing OpenTelemetry no Jaeger com um único trace do `POST` até a chamada ao provedor	`application.yml` · `TracingIT`, `ScheduledProcessingIT`
10. Cenário de análise	Desc.	Investigação, métricas, hipótese de causa comum e validação da correção	README · Cenário de análise
11. Docker	Impl.	Multi-stage, imagem JRE enxuta, não-root, memória ajustada ao container, desligamento gracioso, health check, compose completo	`Dockerfile`, `docker-compose.yml`
Microsserviços e cloud	Desc.	Várias instâncias, segredos, eventos via outbox, pipeline de CI/CD	README · Produção
Entrega	Impl.	README com todas as seções obrigatórias, ADRs e uso de IA	`README.md`, `docs/adr`

Além do pedido: Swagger UI com login OAuth2 real, coleção Postman com verificações automáticas, gate de 100% de cobertura no build e especificação rastreável em `docs/spec`.

4. Arquitetura

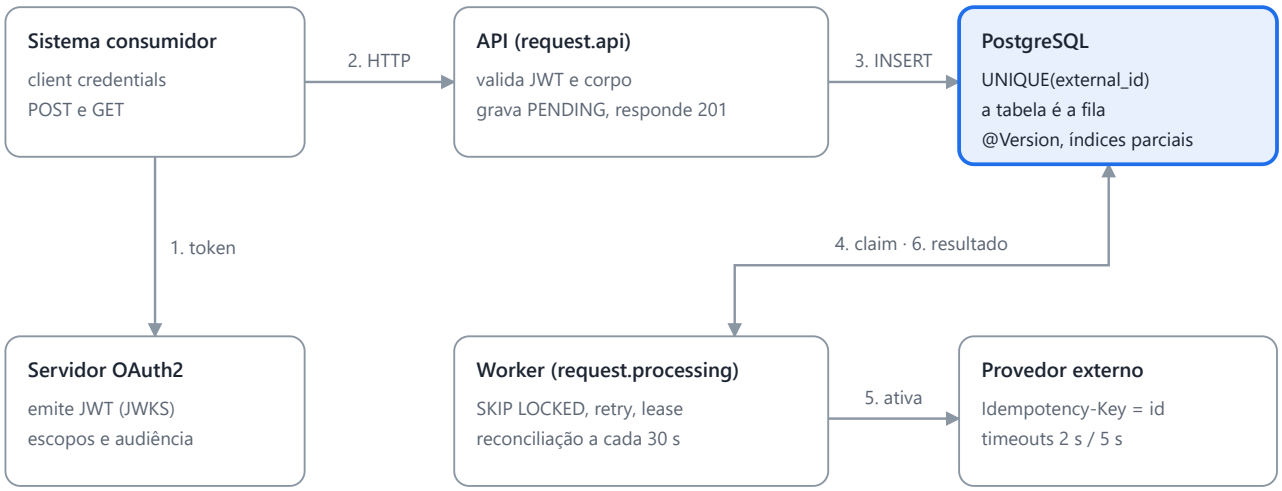


Figura 1 — componentes e quem fala com quem.

Organização dos pacotes

Um pacote por funcionalidade (`request`), com camadas leves dentro; código transversal em `shared`; o cliente do provedor isolado em `provider`.

Pacote	Responsabilidade	Regra (verificada pelo ArchUnit)
`request.domain`	Entidade, estados, regras de transição	Só depende do JDK e de `jakarta.persistence`
`request.api`	Controller, DTOs (records), contrato HTTP	Nunca acessa repositórios
`request.application`	Casos de uso: criar (duplicidade), consultar, listar	—
`request.persistence`	Repositório, queries de fila, specifications	—
`request.processing`	Worker, retry, circuit breaker, reconciliação, métricas	—
`provider`	HTTP com o provedor, classificação das respostas	Não conhece nada de `request`
`shared`	Erros, segurança e limite de chamadas, correlação, tracing, filtros, OpenAPI, relógio	Não depende de nenhuma feature

Por que não hexagonal? O enunciado pede explicitamente para não exagerar. Pacote por funcionalidade dá coesão (tudo de "solicitação" junto) e as regras do ArchUnit garantem o essencial da arquitetura limpa — domínio independente e dependências apontando para dentro — sem interfaces de porta para cada adaptador.

5. Ciclo de vida de uma solicitação

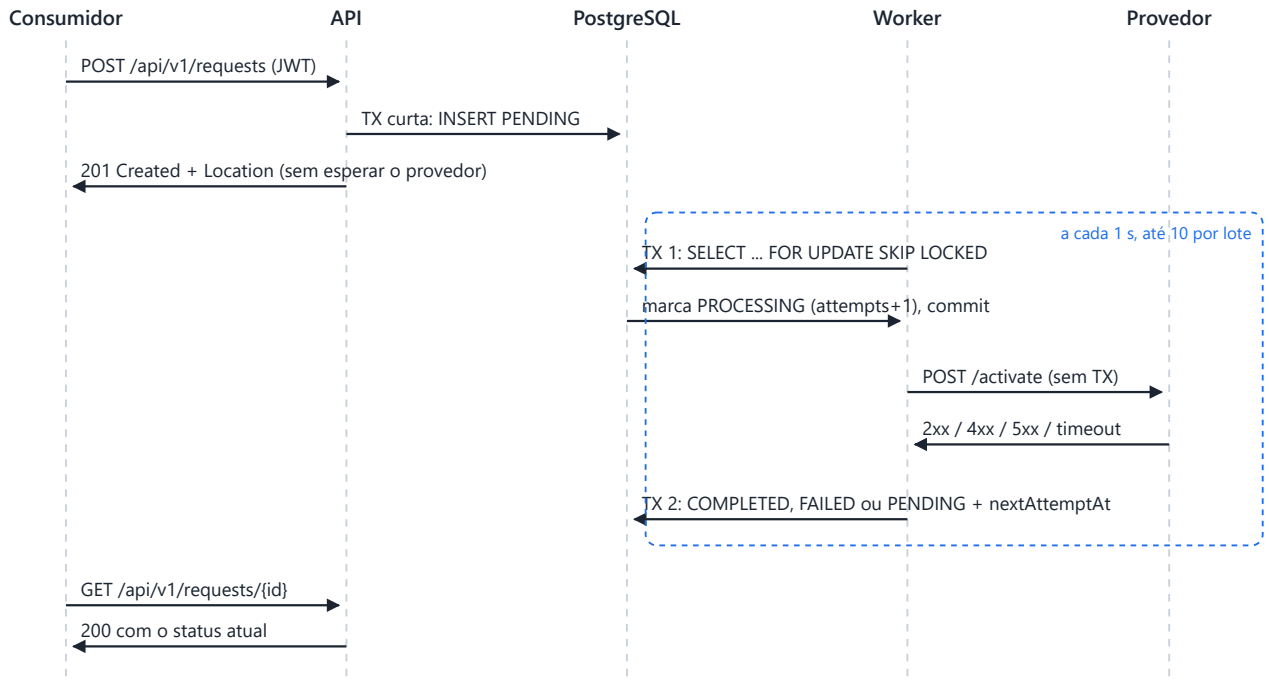


Figura 2 — sequência completa. Repare nas duas transações curtas com a chamada HTTP no meio, fora de qualquer transação.

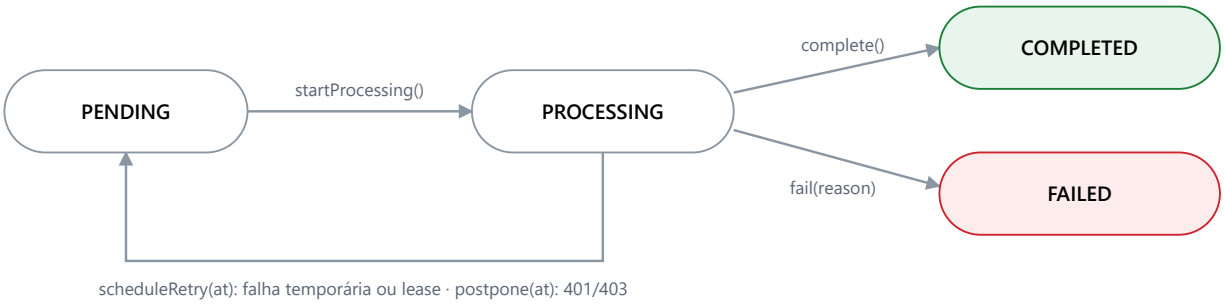


Figura 3 — máquina de estados. Só os métodos do domínio mudam o status; qualquer outra transição lança `IllegalStatusTransitionException`.

Invariantes que o sistema garante

- **Unicidade:** nunca existem dois registros com o mesmo `externalId` — garantido pelo banco, não pelo código.
- **Nada se perde:** um pedido aceito (201) está no banco; se a instância cair, outra o processa.
- **Nunca em paralelo:** duas instâncias não chamam o provedor ao mesmo tempo para o mesmo pedido (`SKIP LOCKED`).
- **Sem ativação dupla:** repetições ao provedor levam a mesma `Idempotency-Key`.
- **Estados finais são finais:** `COMPLETED` e `FAILED` não mudam mais.
- **Queda do provedor não consome tentativas:** com o circuit breaker aberto, nada sai da fila.
- **Resultado atrasado não sobrescreve:** cada claim carrega o número da tentativa; resposta de uma tentativa antiga é descartada.

6. Decisões técnicas e trade-offs

As quatro decisões estruturais têm ADR em `docs/adr`, com contexto e consequências completos; abaixo, o resumo de cada uma e das decisões menores.

D1. Processamento assíncrono com a tabela como fila ADR-0001

Problema	O provedor pode ficar lento ou fora do ar; o pedido gravado não pode se perder se a aplicação cair entre o commit e a chamada.
Decisão	POST grava `PENDING` e responde; worker agendado reivindica lotes com `FOR UPDATE SKIP LOCKED`, chama o provedor fora de transação e grava o resultado.
Alternativas	Síncrono (latência acoplada ao provedor); `@Async` após o commit (evento em memória se perde); Kafka + outbox (infraestrutura extra sem ganho nesta escala).
Por que	Entrega *at-least-once* garantida pelo próprio banco, várias instâncias sem coordenação, nenhuma peça nova de infraestrutura.
Custo aceito	O cliente precisa consultar o status (polling); até 1 s de latência até a primeira tentativa; carga leve e constante de polling no banco.

D2. Duplicidade pela constraint única ADR-0002

Problema	O sistema de origem faz retry; duas requisições iguais podem chegar ao mesmo tempo, em instâncias diferentes.
Decisão	`UNIQUE(external_id)`; inserir direto e, na violação *dessa* constraint, reler fora da transação que falhou: mesmo conteúdo → 200; conteúdo diferente → 409; outro cliente → 409 genérico.
Alternativas	Verificar antes de inserir (falha em concorrência); `ON CONFLICT DO NOTHING` nativo (preso ao PostgreSQL); lock pessimista ou advisory lock (contenção desnecessária); 409 para toda repetição (ruim para quem faz retry).
Por que	Correto sob qualquer concorrência e número de instâncias; o caso comum custa um único INSERT.
Custo aceito	O método de criação não pode ser `@Transactional`; unicidade é global (não por cliente).

D3. Retry persistido, falha temporária volta para PENDING ADR-0003

Problema	Distinguir erro definitivo de temporário; o timeout é ambíguo (o provedor pode ter ativado).
Decisão	4xx → FAILED, exceto 401/403 (credencial nossa: adia 1 min sem gastar tentativa); 408, 5xx, timeout, I/O → PENDING com backoff exponencial e full jitter; 429/503 respeitam `Retry-After`; 5 tentativas no máximo; `Idempotency-Key` em toda chamada; circuit breaker abre com falhas do provedor e pausa o worker sem gastar tentativas.
Alternativas	Retry em memória (Spring Retry/Resilience4j) prende a thread e se perde num restart; manter em PROCESSING com contador mistura "em andamento" com "aguardando".
Por que	O retry sobrevive a reinícios e se distribui entre instâncias; PROCESSING antigo vira sinal inequívoco de instância caída.
Custo aceito	Depende de o provedor honrar a chave de idempotência; FAILED por esgotamento exige ação operacional.

D4. OAuth2 Resource Server com isolamento por cliente ADR-0004

Problema	API consumida por sistemas; separar quem cria de quem consulta; impedir que um sistema leia dados de outro.
Decisão	Client credentials; JWT validado localmente (assinatura via JWKS, `iss`, `aud`, `exp`); escopos `activation:write` / `read`; dono = `client_id` do token, filtrado em toda consulta; outro dono → 404.
Alternativas	API key estática (sem expiração nem escopos); introspecção do token a cada chamada (latência extra); mTLS (complementar, não substitui autorização).
Custo aceito	Revogação só na expiração do token (mitigado por tokens curtos).

Outras decisões

Decisão	Justificativa curta
Spring Boot 3.5 (e não 4.x)	O enunciado pede 3.x, provável stack da empresa. O suporte OSS da 3.5 terminou em jun/2026; a migração está nas melhorias futuras.
PostgreSQL também nos testes	Unicidade concorrente, `SKIP LOCKED` e `TIMESTAMPTZ` se comportam diferente no H2 — passaria no teste e falharia em produção.
200 para retry, 409 para conflito	Retry é normal em integração; o cliente não deveria tratar erro como sucesso. O 200 (e não 201) indica que nada foi criado.
404 (não 403) para dado de outro cliente	403 confirmaria que o id existe (OWASP API1 — BOLA).
id malformado na URL → 404	Um identificador que não pode existir é um recurso inexistente; filtro inválido na query é 400.
`RestClient` sobre `HttpClient` do JDK	Síncrono, moderno, timeouts explícitos, sem trazer WebFlux; criado a partir do builder do Spring para ganhar métricas.
Relógio injetável com precisão de ms	Testes determinísticos; o PostgreSQL guarda microssegundos e arredondava nanos, fazendo o POST e o GET devolverem datas diferentes.
Mensagens de validação em inglês e explícitas	Sem isso, o Bean Validation responde no idioma da JVM — português numa máquina, inglês no container.
Actuator em porta separada	Health e métricas acessíveis ao orquestrador e ao Prometheus, nunca expostos publicamente.
`open-in-view: false`	Nenhuma conexão de banco presa enquanto a resposta é serializada.

7. Estratégia de testes

Nível	O que prova	Ferramentas / exemplos
Unitário puro	Regras sem infraestrutura: transições, retry, classificação de respostas, parsing	JUnit, AssertJ — `ActivationRequestTest`, `RetryPolicyTest`
Unitário com mocks	Decisões de orquestração: ramos da duplicidade, isolamento de erros no worker	Mockito — `ActivationRequestServiceTest`, `ActivationWorkerTest`
Slice web	Contrato HTTP, validação, formato de erro, 401/403/413/429	`@WebMvcTest`, MockMvc, `jwt()`
Integração	Comportamentos que dependem do banco e da rede real	Testcontainers (PostgreSQL), WireMock, Awaitility
Arquitetura	Regras de dependência e ausência de ciclos	ArchUnit

Testes que merecem atenção

- **Duplicidade concorrente:** 20 threads liberadas juntas por um `CountDownLatch`; 19 batem na constraint — a corrida é real.
- **Prova do SKIP LOCKED:** um worker segura 4 linhas numa transação aberta; o outro pega só as 2 restantes, sem esperar.
- **JWT real:** chaves RSA geradas no teste, JWKS publicado num WireMock, tokens expirados/forjados/de outra audiência recusados.
- **Resultado atrasado:** claim antigo tentando gravar depois de o pedido ter sido reprocessado — descartado.
- **Fluxo ponta a ponta:** agendador real ligado, Awaitility esperando o `COMPLETED`, verificação do header de correlação no provedor.
- **Trace atravessando o assíncrono:** com tracing ligado, o `traceparent` gravado no `POST` chega ao provedor na chamada feita depois pelo worker.

Cobertura de 100%

O build falha abaixo de 100% de linhas e ramos (JaCoCo, só o `main()` excluído). O objetivo não foi o número: ao fechar as lacunas, **código defensivo inatingível foi removido** em vez de testado artificialmente, e os testes novos **encontraram um bug real**. Num sistema grande, um gate um pouco menor somado à revisão de testes no code review seria mais saudável.

Defeitos encontrados durante o desenvolvimento

Bug	Como apareceu e como foi resolvido
POST e GET com datas diferentes	Revisão contra o enunciado; nanos em memória × micros no banco → relógio com precisão de ms + teste
`fail(null)` mudava o status antes de lançar	Teste novo de cobertura; validação movida para antes da mudança de estado
Header com caractere de controle virava 500	Teste de log injection; firewall do Spring Security → 400 em ProblemDetail
Ciclo de pacotes `shared ↔ request`	ArchUnit; exceções-base em `shared.error`
Teste instável com WireMock	Primeira chamada a um servidor frio estourava o timeout e gerava retry legítimo; timeouts de teste ajustados e asserções filtradas pela chave de idempotência

8. Operação e produção

Tema	Como está
Imagem Docker	Multi-stage, JRE Alpine, camadas do Spring Boot, usuário não-root, `MaxRAMPercentage=75`, `ExitOnOutOfMemoryError`, `exec` para receber SIGTERM, HEALTHCHECK
Ambiente local	`docker compose`: app, PostgreSQL, WireMock (provedor com cenários), mock-oauth2-server e Jaeger
CI	GitHub Actions: `./mvnw verify` (testes, arquitetura, cobertura) + build da imagem
Configuração	Só variáveis de ambiente; segredos de um cofre (Key Vault / Secrets Manager)
Escala	Várias réplicas sem coordenação; worker pode ser separado da API com `activation.processing.enabled`
Desligamento	Gracioso: requisições em andamento terminam; o agendador espera até 30 s pela chamada em curso
Observabilidade	Probes, métricas Prometheus (provedor por resultado, transições, fila atual por status, HTTP com histograma), logs JSON com `traceId` e `correlationId`, tracing OpenTelemetry no Jaeger atravessando a etapa assíncrona
Documentação viva	OpenAPI em `/v3/api-docs` e Swagger UI com login OAuth2 real

9. Limitações conhecidas e próximos passos

Pontos que ficaram de fora ou que merecem evolução, com o caminho proposto para cada um:

Limitação	Impacto	Como evoluir
Unicidade de `externalId` global	Dois sistemas de origem não podem usar o mesmo id	`UNIQUE(client_id, external_id)`
Consumidor precisa fazer polling	Carga e latência de descoberta	Evento `ActivationCompleted` via outbox + Kafka, ou webhook
Sem reprocessamento de `FAILED`	Operação manual no banco	Endpoint administrativo com escopo próprio e auditoria
Limite de chamadas por instância	Com N instâncias, um cliente pode fazer até N vezes o limite	Limite global no API Gateway (ou contador compartilhado, como Redis)
Spring Boot 3.5 sem suporte OSS	Sem patches gratuitos	Migrar para 4.x (Jackson 3, Hibernate 7)
Tabela cresce sem limite	Volume e custo ao longo do tempo	Particionamento ou arquivamento por data
403 do provedor tratado como credencial recusada	Se o provedor usar 403 para recusa de negócio, o pedido nunca vira `FAILED`	Combinar o contrato com o provedor e, se for o caso, devolver o 403 à regra de `FAILED`
`FAILED` após timeouts sucessivos	O provedor pode ter ativado; o status não reflete isso	Consultar o provedor pela `Idempotency-Key` antes de declarar falha, ou status `UNKNOWN`
Worker chama o provedor uma solicitação por vez	Cerca de 20 ativações/min por instância com o provedor a 3 s	Mais instâncias (`SKIP LOCKED`) ou paralelismo limitado dentro do lote
Paginação por offset com `count(*)`	Páginas profundas ficam caras	Paginação por cursor (`createdAt` , `id`)

10. Guia de leitura do código

Sugestão de ordem para percorrer o código; cada arquivo concentra uma ideia.

#	Arquivo	O que observar
1	`db/migration/V1__create_activation_request.sql`	Constraint única, `CHECK` de status, índices parciais para a fila e compostos para a listagem
2	`request/domain/ActivationRequest.java`	Sem setters; cada transição valida o estado; `@Version`; `fail` valida antes de mudar
3	`request/application/ActivationRequestService.java`	`create`: insere primeiro, decide na violação; por que **não** é `@Transactional`
4	`request/api/ActivationRequestController.java`	`switch` sobre o `sealed CreationResult`: 201 + Location ou 200
5	`request/persistence/ActivationRequestRepository.java`	Queries nativas com `FOR UPDATE SKIP LOCKED`
6	`request/processing/ActivationWorker.java`	Um erro não para o lote; circuit breaker; span que continua o trace do `POST`; timer por resultado
7	`request/processing/ActivationProcessingService.java`	As duas transações curtas; descarte de resultado atrasado; reconciliação
8	`provider/ProviderResponses.java` e `ProviderClient.java`	A tabela do item 5 em código; `sealed ProviderResult`; nunca lança exceção
9	`request/processing/RetryPolicy.java`	Backoff, jitter injetável, Retry-After, limite
10	`shared/security/SecurityConfiguration.java` e `CurrentClient.java`	Escopos por rota; fallback `client_id → azp → sub`
11	`shared/error/ApiExceptionHandler.java`	Todo erro em ProblemDetail, inclusive os do próprio Spring
12	`application.yml`	Cada propriedade tem um motivo; os comentários explicam

Recursos de Java moderno usados (e onde)

Recurso	Onde
`record`	DTOs, comandos, `ProviderResult`, propriedades
`sealed interface`	`CreationResult`, `ProviderResult`
Pattern matching no `switch`	Controller, processamento, métricas

Recurso	Onde
Text blocks	Queries nativas, testes
`var`	Variáveis locais em todo o código
`@ConfigurationProperties` com record	`ProviderProperties`, `RetryProperties`, `CircuitBreakerProperties`, `RateLimitProperties`

O ganho do `sealed` + `switch`: quando um novo tipo de `ProviderResult` é criado, o compilador aponta todo `switch` que não o trata. Foi o que aconteceu ao incluir `CredentialsRejected` (401/403): os dois `switch` que precisavam tratá-lo pararam de compilar até serem ajustados. É segurança em tempo de compilação para a regra de negócio mais sensível.

11. Processo de desenvolvimento e uso de IA

O projeto foi desenvolvido com o **Claude Code** (Anthropic) como ferramenta, dentro de um processo de **Spec-Driven Development**. A IA aumentou a velocidade de execução; o processo garantiu o controle sobre as decisões e a qualidade.

1. **Análise** do enunciado e da vaga: obrigatórios × diferenciais; o que implementar e o que descrever.
2. **Especificação antes do código** (`docs/spec`, `docs/adr`): requisitos com critérios de aceite, design e plano; alternativas comparadas para cada decisão estrutural e aprovadas antes da implementação.
3. **Implementação incremental** em cinco fases, com commits que citam a tarefa do plano (ligada aos requisitos em `tasks.md`) e um checkpoint de revisão ao fim de cada fase.
4. **Verificação** por testes automatizados (critério de pronto), execução real da stack e revisões contra o enunciado.

Responsabilidade	Desenvolvedor	IA (Claude Code)
Escopo e prioridades	Definiu	Propôs opções
Decisões técnicas e trade-offs	Escolheu e aprovou	Levantou alternativas, prós e contras
Especificação e ADRs	Revisou e aprovou	Redigiu a partir das decisões
Código, testes e infraestrutura	Revisou a cada checkpoint	Implementou, tarefa por tarefa
Critérios de qualidade	Definiu (cobertura total, plano de QA, revisão contra o enunciado)	Executou e reportou
Validação final	Testou manualmente a aplicação	Rodou builds, testes e verificações

As revisões encontraram defeitos que uma geração direta de código teria deixado passar (seção 7). O detalhamento está na seção **Uso de IA** do README.